

Presentación

El objetivo de esta asignatura es presentar la mención de Ingeniería de Software al estudiante del Grado de Ingeniería Informática. Se introducirán algunas técnicas y metodologías para la construcción de productos software que el alumno podrá ampliar en futuras asignaturas del grado:

- El análisis y diseño del software será estudiado en profundidad en la asignatura “*Análisis y Diseño del Software*” en tercer curso del grado.
- La verificación y validación de los productos software será indagada en la asignatura “*Verificación y Validación del Software*” en tercer curso del grado.
- La arquitectura del software será analizada en detalle en la asignatura “*Arquitectura del Software*” en tercer curso.
- Las metodologías de construcción de software, tanto la visión predictiva, como sobre todo la visión adaptativa, serán estudiadas en la asignatura “*Gestión de Proyectos I*” en cuarto curso.
- Las técnicas de programación que permiten minimizar las vulnerabilidades de seguridad de los productos software serán estudiadas en la asignatura “*Programación Segura*” de tercer curso.
- Las tecnologías específicas con las que construir productos web serán indagadas en la asignatura “*Ingeniería Web*” del tercer curso.
- El diseño de un sistema específico de *business intelligence* se explicará en la asignatura “*Análisis de Aplicaciones Empresariales*” de tercer curso.
- Las principales herramientas de apoyo al desarrollo como el Control de Versiones o la Integración Continua serán tratadas en la asignatura “*Gestión de Proyectos II*” en el último curso.

Además, los alumnos del grado, independientemente de la mención elegida, deberán profundizar en la programación orientada a objetos en la asignatura “*Programación Avanzada*” en el segundo semestre del segundo curso.

Esta asignatura tratará todas estas cuestiones de forma divulgativa, sin profundizar en los detalles, de esta forma el alumno obtendrá una visión general y superficial de la profesión de ingeniero de software.

Teoría

En el primer tema, "Panorámica", el alumno tendrá su primer contacto con la Programación Orientada a Objetos. Además se mostrará brevemente la historia de la Ingeniería del Software, el Ciclo de Vida del Software y diferentes visiones de construcción. El primer tema introducirá conceptos esenciales como la importancia de los diagramas y de las herramientas CASE en la ingeniería. Los siguientes capítulos se alinean con cada una de las actividades del ciclo de vida del software: Requisitos, Análisis, Diseño, Codificación, Pruebas y Despliegue.

- Tema 1. Panorámica
- Tema 2. Requisitos
- Tema 3. Análisis
- Tema 4. Diseño
- Tema 5. Codificación
- Tema 6. Pruebas
- Tema 7. Despliegue

Por último, se explicará la visión económica de la construcción de software, así como diversas metodologías de construcción:

- Tema 8. Economía
- Tema 9. Metodologías

Cada tema se acompañará de los apuntes y las diapositivas utilizadas por el docente. Todo el material se publicará en *miAulario*. Los apuntes contienen apartados destacados con el símbolo  para las citas, con el símbolo  para ampliaciones de contenido y con el símbolo  para ejemplos.

Para ayudar a entender lo explicado se realizarán, tanto en grupo como individualmente, algunos ejercicios en clase. Los ejercicios se complementarán con material audiovisual y lecturas recomendadas.

Prácticas

En las prácticas el alumno aprenderá a programar siguiendo el paradigma orientado a objetos haciendo uso de Java. Al principio adquirirá los fundamentos del lenguaje, al final pondrá en práctica los conceptos explicados en teoría construyendo un producto software.

- Práctica 1. Introducción a Java.
- Práctica 2. Sintaxis básica.
- Práctica 3. Clases y Objetos.
- Práctica 4. Interface y Herencia.
- Práctica 5. Netbeans. El alumno aprenderá a usar un IDE.
- Práctica 6. Green Garden. El alumno aprenderá a describir formalmente requisitos haciendo uso de Historias de Usuario. También aprenderá a dibujar diagramas de casos de uso, de clases y de secuencia con ayuda de una herramienta de diseño gráfico.
- Práctica 7. Clean Code. Es posible implementar una misma funcionalidad con muy diferentes códigos. No todos los códigos ofrecen la misma facilidad para su mantenimiento y modificación. En esta práctica el alumno deberá "refactorizar" código para hacerlo más fácil de entender.

- Práctica 8. El Hospital. El alumno aprenderá a escribir tests haciendo uso de un framework de pruebas, también implementará una versión simplificada del patrón MVP.
- Trabajo Final. En el trabajo final de la asignatura el alumno deberá hacer uso de todo lo aprendido durante las prácticas para construir un producto software.